Introduction

This worksheet covers the **Self-Attention algorithm** we looked at in ArchitectureⅡ. Similar to last week, you will do some work implementing your own versions of these algorithms, to ensure that you understand the details of them.

Step by step coding

We will code Self-Attention in PyTorch step by step.

1. Import key packages: torch, and any others that you prefer to work with. In general, when writing code, you will put all your import statements at the top. However, for these worksheets we will import as we go along.

In [ ]:
```
import torch
```

2. Set the number of embedding values per token, in Architecture Ⅱ.

In [ ]:
```
model_dim= 2
```

3. Initialize the Linear Layers that we will use to create the Query, Key and Value. W_q, W_k and W_v represents the weights of Query, Key and Value, seperately.

In [ ]:
```
W_q_linear = torch.nn.Linear(in_features=model_dim, out_features=model_dim, bi
W_k_linear = torch.nn.Linear(in_features=model_dim, out_features=model_dim, bi
W_v_linear = torch.nn.Linear(in_features=model_dim, out_features=model_dim, bi
```

4. Create a matrix of encoded_values as the input of self-attention module

In [ ]:
```
token_len= 2
encoded_values = torch.randn(token_len, model_dim)
```

5. Create the query, key and values using the encoded values through corresponding linear layers.

In [ ]:
```
q= W_q_linear(encoded_values)
k= W_k_linear(encoded_values)
v= W_v_linear(encoded_values)
```

6. Multiply the Query matrix by the transpose of the Key matrix to compute the similarities scores.

In [ ]:
```
scores = torch.matmul(q, k.transpose(-1,-2))
```

7. Scale the matrix of dot product similarities.

In [ ]:
```
scores = scores / (k.shape[-1]**0.5)
```

8. Take the SoftMax of each row in the matrix of scaled Dot Product similarities.

In [ ]:
```
attn = torch.nn.functional.softmax(scores, dim=-1)
```

9. Multiply by the Values in matrix V to scale the values by their associated percentages and add them up.

In [ ]:
```
attn = torch.matmul(attn, v)
# print out the attn output
print(attn)
```

Wrapped inside a self-attention class

We can organize and encapsulate the code above into a self-attention class with both initialization and forward computation.

In [ ]:
```
class SelfAttention(torch.nn.Module):

    def __init__(self, model_dim=2):
        super().__init__()
        # Initialize the Linear Layers that we will use to create the Query, K
        # W_q, W_k and W_v represents the weights of Query, Key and Value, sep
        self.W_q_linear= torch.nn.Linear(in_features=model_dim, out_features=m
        self.W_k_linear= torch.nn.Linear(in_features=model_dim, out_features=m
        self.W_v_linear= torch.nn.Linear(in_features=model_dim, out_features=m
        self.model_dim=model_dim

    def forward(self, encoded_values):
        ## Create the query, key and values using the encoded values through c
        q = self.W_q_linear(encoded_values)
        k = self.W_k_linear(encoded_values)
        v = self.W_v_linear(encoded_values)

        ## Multiply the Query matrix by the transpose of the Key matrix to com
        scores = torch.matmul(q, k.transpose(-1,-2))

        ## Scale the matrix of dot product similarities.
```

```
        scores = scores/(k.shape[-1]**0.5)

        ## Take the SoftMax of each row in the matrix of scaled Dot Product si
        attn = torch.nn.functional.softmax(scores, dim=-1)

        ## Multiply by the Values in matrix V to scale the values by their ass
        attn = torch.matmul(attn, v)

        return attn
```

Use this Self-Attention Class

In [ ]:

```
# set the token_len and create a random encoded_values
token_len= 2
encoded_values = torch.randn(token_len, model_dim)
# create a self-attention ojbect
selfAttention = SelfAttention(model_dim=2)
# calculate attention for the token encodings
attn=selfAttention(encoded_values)
# print out the attn output
print(attn)
```